Lab Preliminary Report

Course: Cs 223 Digital Design

Section: 2

Lab Number: 3

Name: Eralp Yiğit Boz

ID: 22403188

Date: 13.04.2026

Queue Depth Justification: A 3-bit queue counter was chosen, allowing a maximum queue depth of 7 (plus 1 currently processing, totaling 8 requests per machine). Practically, having more than 7 people waiting for a single machine is highly unlikely. Technically, using a 3-bit counter instead of a larger one minimizes the use of FPGA resources (Flip-Flops and LUTs), resulting in a more hardware-efficient design.

Question 1) Indicate which type of FSM (Moore or Mealy) you will implement and justify your choice.

I believe Moore is the most suitable type for this system. I have two reasons for this:

1) Output Stability: In Mealy machines, outputs are directly dependent on instantaneous inputs. This means that if there are glitches in the inputs, this error will be directly reflected in the FSM outputs. In the system we built, the FSM outputs trigger the Timer and Queue in the Datapath. If I had designed it with Mealy, millisecond delays in the buttons could have broken the system. In a Moore Machine, since the outputs are only dependent on registers, small delays will not cause problems.

2) Design Complexity: The lab PDF asks us to shorten the combinational logic part of the system using a decoder. A Moore FSM aligns perfectly with this requirement. Since Moore outputs depend exclusively on the current state, we can connect the state register bits directly to the decoder and use the decoder's output pins as our 'Request' signals to the Datapath without needing any extra logic gates. If a Mealy machine were used, we would have to combine the decoder outputs with the current button inputs using extra AND gates. This would increase the combinational logic and contradict the lab's goal of simplification.

Quesiton 2) Provide the state transition diagram, state encoding, state transition and output tables, next-state and output equations, and the FSM schematic.

2.1) State Encoding

State Name	Q2	Q1	Q0
S0_IDLE	0	0	0
S_REQ_QUICKWASH	0	0	1
S_REQ_HEAVYWASH	0	1	0
S_REQ_DRYER	0	1	1
S_REQ_DELICATEWASH	1	0	0

2.2) State Transition Table

Current State	Pulse_Q	Pulse_H	Pulse_DR	Pulse_DW	Next State
S0_IDLE	1	X	X	X	S_REQ_QUICKWASH
S0_IDLE	0	1	X	X	S_REQ_HEAVYWASH
S0_IDLE	0	0	1	X	S_REQ_DRYER
S0_IDLE	0	0	0	1	S_REQ_DELICATEWASH
S0_IDLE	0	0	0	0	S0_IDLE
S_REQ_QUICKWASH	X	X	X	X	S0_IDLE
S_REQ_HEAVYWASH	X	X	X	X	S0_IDLE
S_REQ_DRYER	X	X	X	X	S0_IDLE
S_REQ_DELICATEWASH	X	X	X	X	S0_IDLE

2.3) Output Table

Current State	Req_Q	Req_H	Req_DR	Req_DW
S0_IDLE	0	0	0	0

S_REQ_QUICKWASH	1	0	0	0
S_REQ_HEAVYWASH	0	1	0	0
S_REQ_DRYER	0	0	1	0
S_REQ_DELICATEWASH	0	0	0	1

2.4) Equations

Current State Bits: Q2, Q1, Q0

Next State Bits: D2, D1, D0

Input Bits: Pq, Ph, Pdr, Pdw

2.4.1) Next State Equations

D2 = Q2' · Q1' · Q0' · Pq' · Ph' · Pdr' · Pdw

D1 = (Q2' · Q1' · Q0' · Pq' · Ph) + (Q2' · Q1' · Q0' · Pq' · Ph' · Pdr)

D0 = (Q2' · Q1' · Q0' · Pq) + (Q2' · Q1' · Q0' · Pq' · Ph' · Pdr)

2.4.2) Output Equations

QuickWash_Request = Q2' · Q1' · Q0

HeavyWash_Request = Q2' · Q1 · Q0'

Dryer_Request = Q2' · Q1 · Q0

DelicateWash_Request = Q2 · Q1' · Q0'

2.5) FSM State Transition Diagram

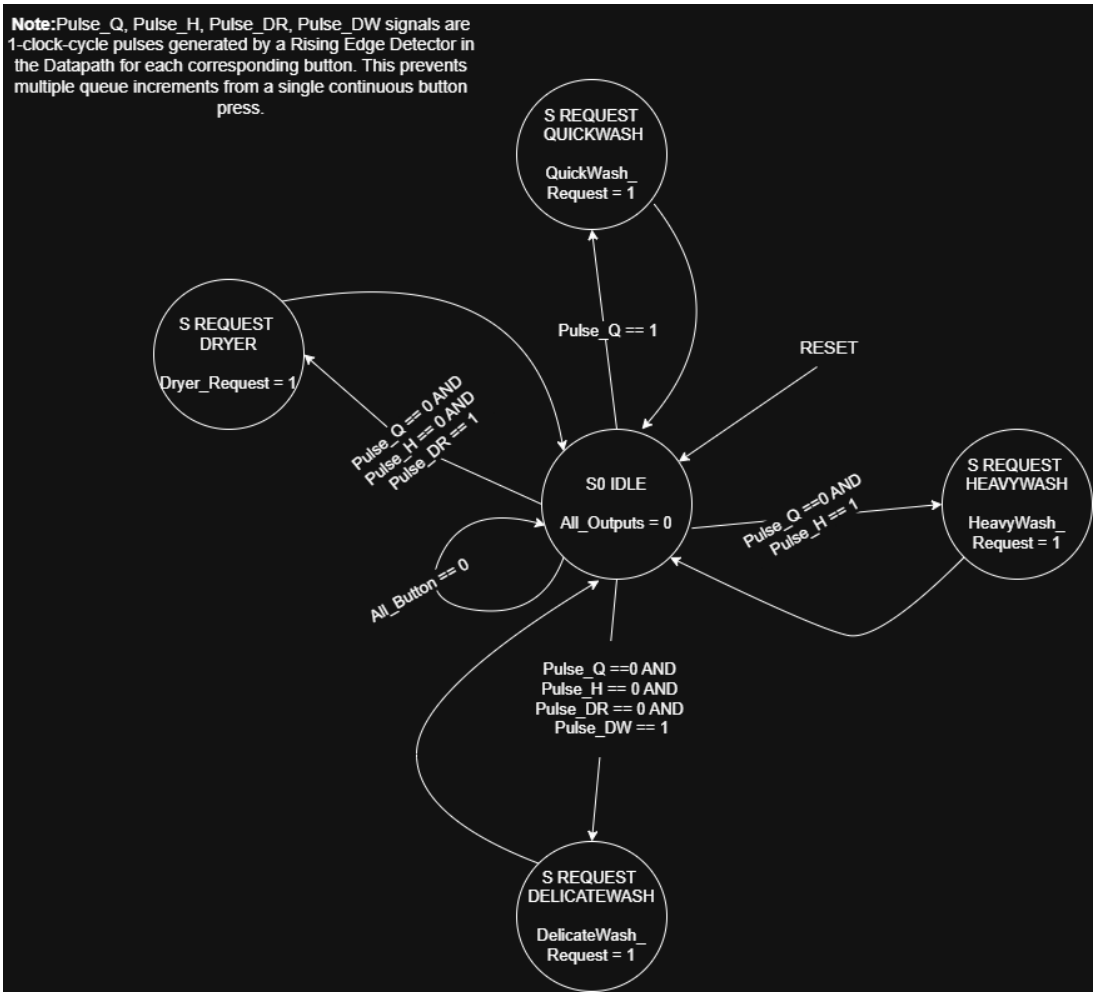

Figure 1. FSM Diagram

Starting from **S0_IDLE**, the FSM processes requests sequentially using fixed priority (Quick > Heavy > Dryer > Delicate). It enters a request state for exactly one cycle to trigger the datapath and then returns to IDLE. This pulse-based logic, combined with edge detection, ensures each button press is handled exactly once, maintaining system stability.

2.6) FSM Schematic

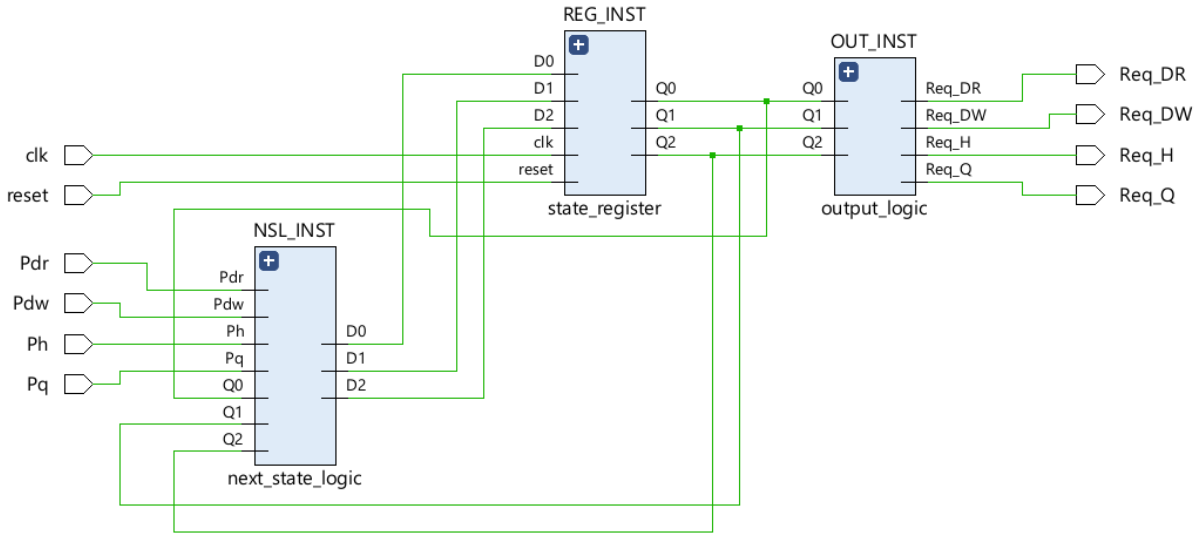

Figure 2. FSM Schematic

2.7) Datapath Schematic

The datapath consists of four parallel units managing machine status, waiting queues, and specific service durations (4s, 7s, 9s, 11s). When a request signal arrives, the machine starts immediately if idle; otherwise, the request increments the queue counter. Integrated feedback loops ensure that the next waiting request starts automatically once a machine completes its cycle, ensuring continuous operation.

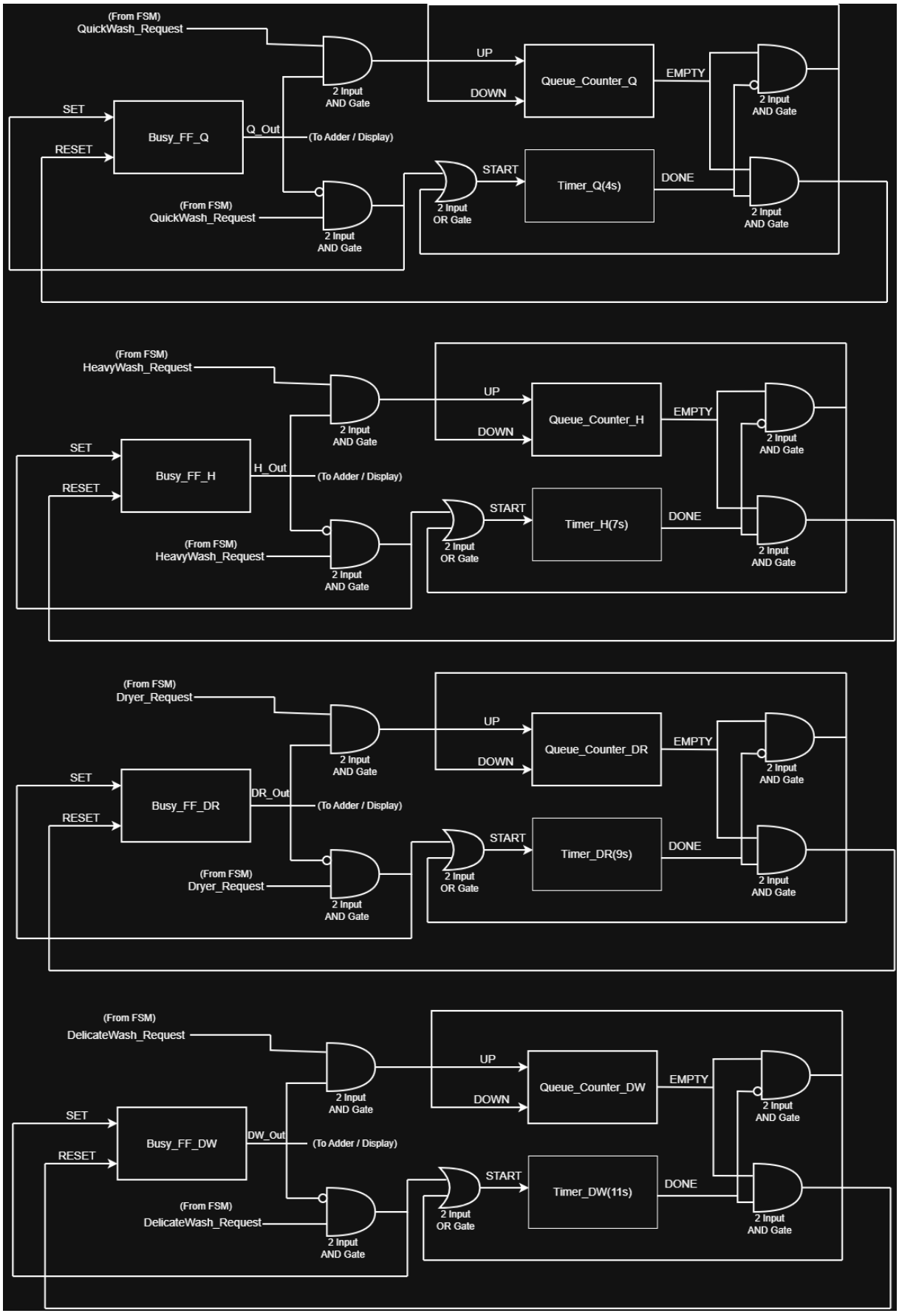

Figure 3. Datapath

Question 3) Determine the minimum number of flip-flops required by your encoding.

The minimum number of flip-flops required for the FSM state encoding is **3**.

Explanation: Our FSM design consists of 5 distinct states. Since 2 flip-flops can only represent 2^2 = 4 states, they are insufficient. Using 3 flip-flops provides 2^3 = 8 possible combinations, which is the minimum number required to successfully encode our 5 states.

It is important to note that these 3 flip-flops are only for the FSM. A significant number of additional flip-flops are required for the datapath Specifically, the datapath contains 4 separate single-bit flip-flops to track the busy status of each machine, as well as many more internal flip-flops located within the Queue Counters and Timers.

Question 4) Redesign your outputs using decoders to simplify the combinational logic. Use a 3-to-8 decoder to generate one-hot encoded state signals from your state register bits, then express your output functions in terms of these decoded signals.

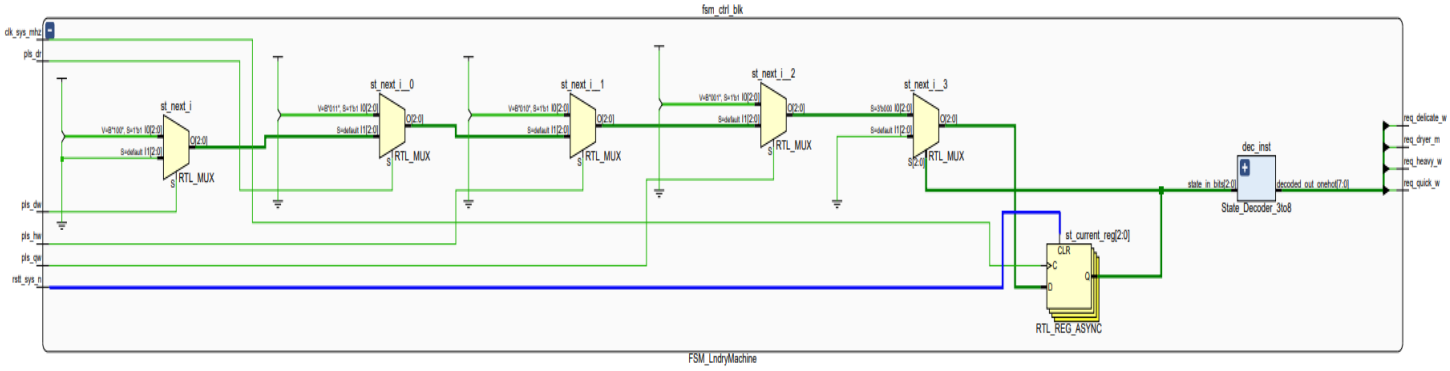

Figure 4. FSM design with decoder

Decoder Output Table

State Name	Encoding (Q2Q1Q0)	Decoder Output (Yn)

S0_IDLE	000	Y0
S_REQ_QUICKWASH	001	Y1
S_REQ_HEAVYWASH	010	Y2
S_REQ_DRYER	011	Y3
S_REQ_DELICATEWASH	100	Y4

QuickWash_Request =Y1

HeavyWash_Request = Y2

Dryer_Request = Y3

DelicateWash_Request = Y4

By using a 3-to-8 decoder, we convert the binary state bits into one-hot encoded signals. This eliminates the need for complex logic gates (AND/OR/NOT) to generate the outputs, as each service request signal can now be directly connected to its corresponding decoder output pin.

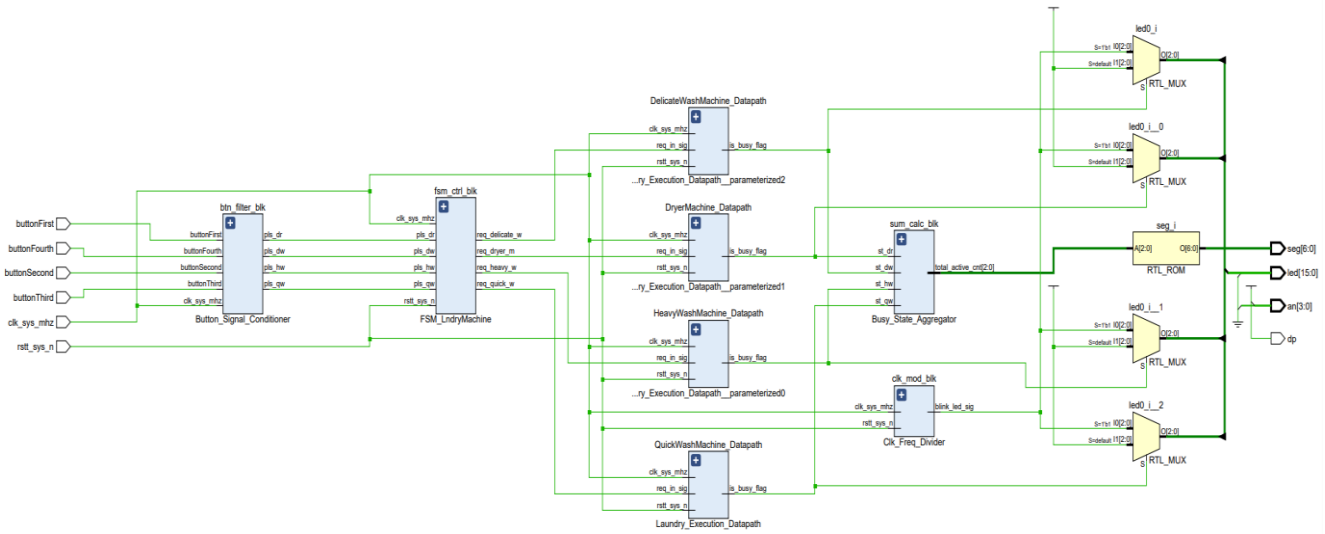

Figure 5. Top-Level RTL Schematic of the Smart Laundry Controller

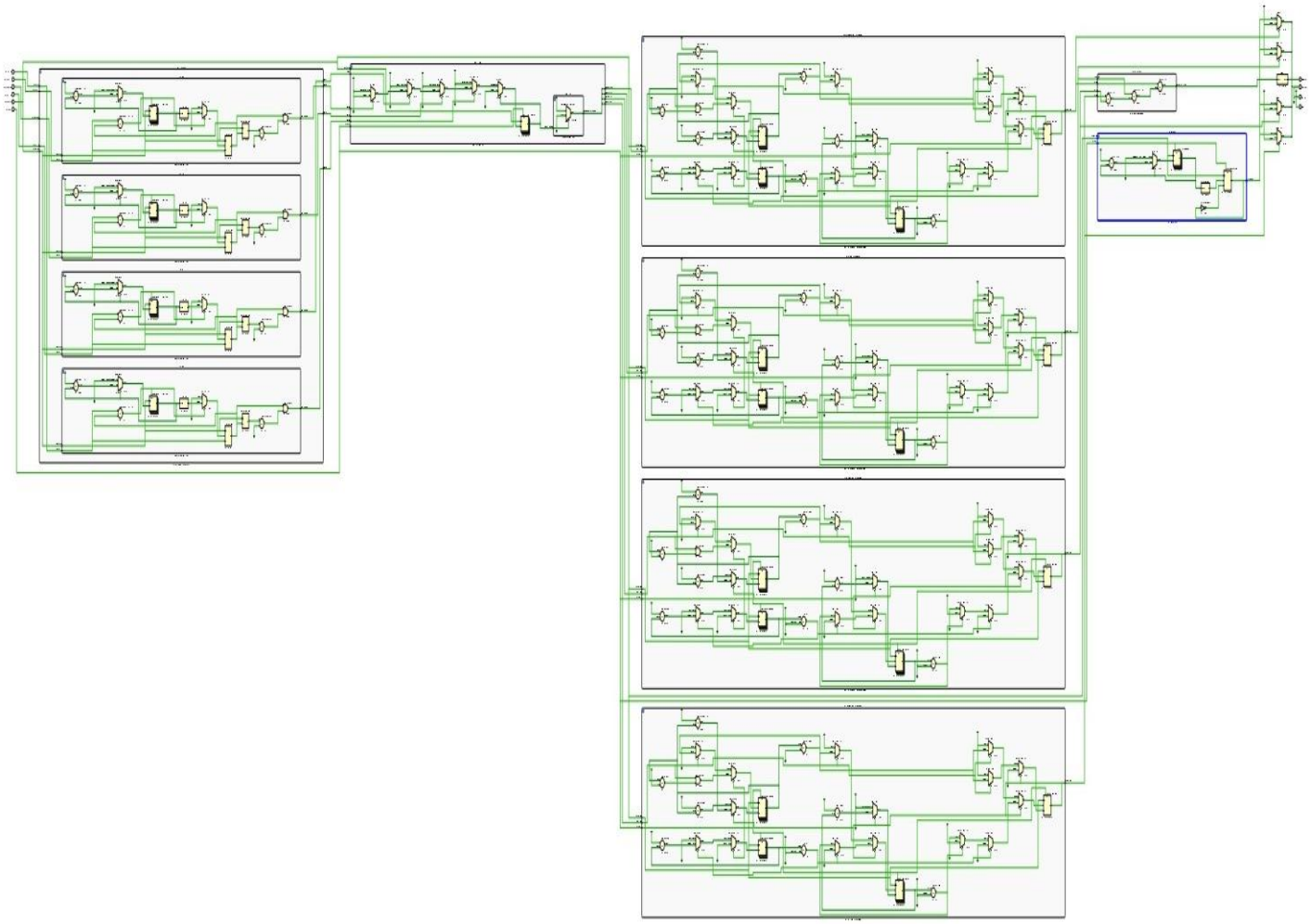

Figure 6. Detailed RTL Schematic of the Smart Laundry Controller

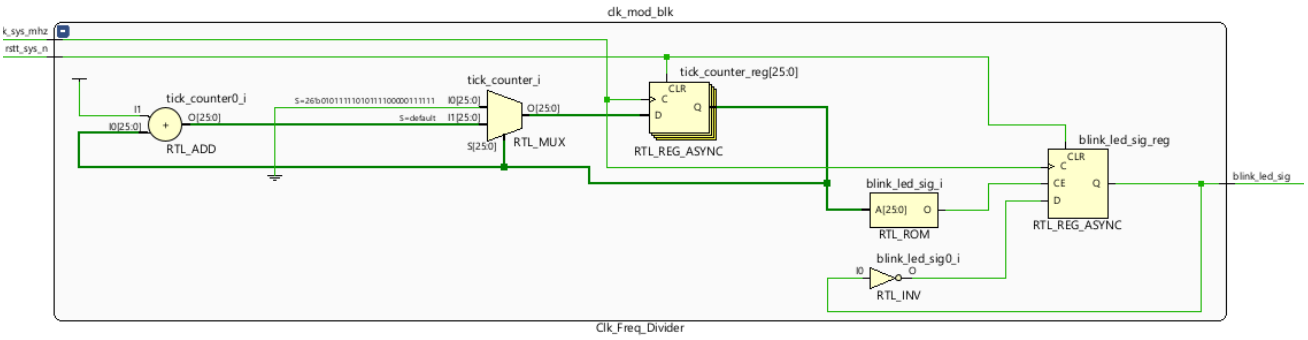

Figure 7. Clock Divider

Question 5) Write a SystemVerilog module implementing your FSM and a self-checking testbench that verifies functionality.

SystemVerilog Module

```
`timescale 1ns / 1ps


// Signal Stabilizer Core
module Signal_Stabilizer_Core (
    input  logic clk_sys_mhz,
    input  logic raw_btn_in,
    output logic clean_pulse_out
);
    logic [19:0] stabilization_cnt;
    logic current_st;
    logic previous_st;

    always_ff @(posedge clk_sys_mhz) begin
        previous_st <= current_st;
        if (raw_btn_in == current_st) begin
            stabilization_cnt <= 0;
        end else begin
            stabilization_cnt <= stabilization_cnt + 1;
            if (stabilization_cnt == 20'd1_000_000) begin
                current_st <= raw_btn_in;
                stabilization_cnt <= 0;
            end
        end
    end
```

```
    assign clean_pulse_out = (current_st == 1'b1) && (previous_st == 1'b0);
endmodule

// Button Signal Conditioner
module Button_Signal_Conditioner (
    input  logic clk_sys_mhz,
    input  logic buttonFirst, buttonSecond, buttonThird, buttonFourth,
    output logic pls_qw, pls_hw, pls_dr, pls_dw
);
    Signal_Stabilizer_Core filter_qw (.clk_sys_mhz(clk_sys_mhz),
.raw_btn_in(buttonFirst),  .clean_pulse_out(pls_qw));
    Signal_Stabilizer_Core filter_hw (.clk_sys_mhz(clk_sys_mhz),
.raw_btn_in(buttonSecond), .clean_pulse_out(pls_hw));
    Signal_Stabilizer_Core filter_dr (.clk_sys_mhz(clk_sys_mhz),
.raw_btn_in(buttonThird),  .clean_pulse_out(pls_dr));
    Signal_Stabilizer_Core filter_dw (.clk_sys_mhz(clk_sys_mhz),
.raw_btn_in(buttonFourth), .clean_pulse_out(pls_dw));
endmodule

// Clock Freq Divider
module Clk_Freq_Divider (
    input  logic clk_sys_mhz, rstt_sys_n,
    output logic blink_led_sig
);
    logic [25:0] tick_counter;
    always_ff @(posedge clk_sys_mhz or posedge rstt_sys_n) begin
        if (rstt_sys_n) begin
            tick_counter <= 0;
```

```
            blink_led_sig <= 0;
        end else begin
            if (tick_counter == 25_000_000 - 1) begin
                tick_counter <= 0;
                blink_led_sig <= ~blink_led_sig;
            end else begin
                tick_counter <= tick_counter + 1;
            end
        end
    end
endmodule

// State Decoder
module State_Decoder_3to8 (
    input  logic [2:0] state_in_bits,
    output logic [7:0] decoded_out_onehot
);
    always_comb begin
        decoded_out_onehot = 8'b00000000;
        decoded_out_onehot[state_in_bits] = 1'b1;
    end
endmodule

// FSM LndryMachine
module FSM_LndryMachine (
    input  logic clk_sys_mhz, rstt_sys_n,
```

```
    input  logic pls_qw, pls_hw, pls_dr, pls_dw,
    output logic req_quick_w, req_heavy_w, req_dryer_m, req_delicate_w
);
    typedef enum logic [2:0] {
        IDLE_ST = 3'b000, REQ_Q_ST = 3'b001, REQ_H_ST = 3'b010,
        REQ_DR_ST = 3'b011, REQ_DW_ST = 3'b100
    } sys_state_t;

    sys_state_t st_current, st_next;

    always_ff @(posedge clk_sys_mhz or posedge rstt_sys_n) begin
        if (rstt_sys_n) st_current <= IDLE_ST;
        else st_current <= st_next;
    end

    always_comb begin
        st_next = IDLE_ST;
        case (st_current)
            IDLE_ST: begin
                if (pls_qw)      st_next = REQ_Q_ST;
                else if (pls_hw)  st_next = REQ_H_ST;
                else if (pls_dr)  st_next = REQ_DR_ST;
                else if (pls_dw)  st_next = REQ_DW_ST;
            end
            default: st_next = IDLE_ST;
        endcase
```

```
    end

    // Decoder Usage
    logic [7:0] decoder_wire;
    State_Decoder_3to8 dec_inst (
        .state_in_bits(st_current),
        .decoded_out_onehot(decoder_wire)
    );

    assign req_quick_w    = decoder_wire[1];
    assign req_heavy_w    = decoder_wire[2];
    assign req_dryer_m    = decoder_wire[3];
    assign req_delicate_w = decoder_wire[4];
endmodule

// Laundry Execution Datapath
module Laundry_Execution_Datapath #(parameter integer PROC_DUR = 4) (
    input  logic clk_sys_mhz, rstt_sys_n, req_in_sig,
    output logic is_busy_flag
);
    logic [2:0] wait_queue;
    logic [3:0] sec_timer;
    logic [26:0] sec_scaler;
    logic tick_1s;

    assign tick_1s = (sec_scaler == 100_000_000 - 1);
```

```
always_ff @(posedge clk_sys_mhz or posedge rstt_sys_n) begin
   if (rstt_sys_n) begin
      is_busy_flag <= 0; wait_queue <= 0; sec_timer <= 0; sec_scaler <= 0;
   end else begin

      if (is_busy_flag) begin
         if (tick_1s) sec_scaler <= 0;
         else sec_scaler <= sec_scaler + 1;
      end else begin
         sec_scaler <= 0;
      end

      if (!is_busy_flag) begin
         if (req_in_sig) begin
            is_busy_flag <= 1;
            sec_timer <= PROC_DUR;
         end else if (wait_queue > 0) begin
            is_busy_flag <= 1;
            sec_timer <= PROC_DUR;
            wait_queue <= wait_queue - 1;
         end
      end else begin
         if (req_in_sig && wait_queue < 7) begin
            wait_queue <= wait_queue + 1;
         end
```

```
            if (tick_1s) begin
               if (sec_timer > 1) begin
                  sec_timer <= sec_timer - 1;
               end else begin
                  is_busy_flag <= 0;
               end
            end
         end

      end
   end
endmodule

// Busy State Aggregator
module Busy_State_Aggregator (
   input logic st_qw, st_hw, st_dr, st_dw,
   output logic [2:0] total_active_cnt
);
   assign total_active_cnt = st_qw + st_hw + st_dr + st_dw;
endmodule

// Main System Wrapper
module laundry_top (
   input  logic clk_sys_mhz, rstt_sys_n,
   input  logic buttonFirst, buttonSecond, buttonThird, buttonFourth,
```

```
    output logic [15:0] led,
    output logic [6:0] seg,
    output logic dp,
    output logic [3:0] an
);
    logic pls_qw, pls_hw, pls_dr, pls_dw;
    logic req_quick_w, req_heavy_w, req_dryer_m, req_delicate_w;
    logic blink_led_sig;
    logic flag_qw, flag_hw, flag_dr, flag_dw;
    logic [2:0] sum_active_mach;

    // Filter Group
    Button_Signal_Conditioner btn_filter_blk (
        .clk_sys_mhz(clk_sys_mhz),
        .buttonFirst(buttonFirst), .buttonSecond(buttonSecond),
        .buttonThird(buttonThird), .buttonFourth(buttonFourth),
        .pls_qw(pls_qw), .pls_hw(pls_hw), .pls_dr(pls_dr), .pls_dw(pls_dw)
    );

    // Clock Mod
    Clk_Freq_Divider clk_mod_blk (
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
.blink_led_sig(blink_led_sig)
    );

    // FSM Controller
    FSM_LndryMachine fsm_ctrl_blk (
```

```
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
        .pls_qw(pls_qw), .pls_hw(pls_hw), .pls_dr(pls_dr), .pls_dw(pls_dw),
        .req_quick_w(req_quick_w), .req_heavy_w(req_heavy_w),
        .req_dryer_m(req_dryer_m), .req_delicate_w(req_delicate_w)
    );

    // QuickWashMachine Datapath
    Laundry_Execution_Datapath #(4) QuickWashMachine_Datapath (
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
.req_in_sig(req_quick_w), .is_busy_flag(flag_qw)
    );

    // HeavyWashMachine Datapath
    Laundry_Execution_Datapath #(7) HeavyWashMachine_Datapath (
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
.req_in_sig(req_heavy_w), .is_busy_flag(flag_hw)
    );

    // DryerMachine Datapath
    Laundry_Execution_Datapath #(9) DryerMachine_Datapath (
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
.req_in_sig(req_dryer_m), .is_busy_flag(flag_dr)
    );

    // DelicateWashMachine Datapath
    Laundry_Execution_Datapath #(11) DelicateWashMachine_Datapath (
        .clk_sys_mhz(clk_sys_mhz), .rstt_sys_n(rstt_sys_n),
.req_in_sig(req_delicate_w), .is_busy_flag(flag_dw)
```

```
);

// RTL Adder Equivalent
Busy_State_Aggregator sum_calc_blk (
   .st_qw(flag_qw), .st_hw(flag_hw), .st_dr(flag_dr), .st_dw(flag_dw),
   .total_active_cnt(sum_active_mach)
);

// LED Outputs
assign led[14:12] = flag_qw ? {3{blink_led_sig}} : 3'b111;
assign led[10:8]  = flag_hw ? {3{blink_led_sig}} : 3'b111;
assign led[6:4]   = flag_dr ? {3{blink_led_sig}} : 3'b111;
assign led[2:0]   = flag_dw ? {3{blink_led_sig}} : 3'b111;
assign led[15] = 0; assign led[11] = 0; assign led[7] = 0; assign led[3] = 0;

// Seven Segment
assign an = 4'b1110;
assign dp = 1;

always_comb begin
   case (sum_active_mach)
      3'd0: seg = 7'b1000000;
      3'd1: seg = 7'b1111001;
      3'd2: seg = 7'b0100100;
      3'd3: seg = 7'b0110000;
      3'd4: seg = 7'b0011001;
```

```
            default: seg = 7'b1111111;
        endcase
    end
endmodule
```

Testbench 1

```
`timescale 1ns / 1ps

module tb_FSM_LndryMachine();


    logic clk_sys_mhz;
    logic rstt_sys_n;
    logic pls_qw, pls_hw, pls_dr, pls_dw;
    logic req_quick_w, req_heavy_w, req_dryer_m, req_delicate_w;


    FSM_LndryMachine uut (
        .clk_sys_mhz(clk_sys_mhz),
        .rstt_sys_n(rstt_sys_n),
        .pls_qw(pls_qw), .pls_hw(pls_hw), .pls_dr(pls_dr), .pls_dw(pls_dw),
        .req_quick_w(req_quick_w), .req_heavy_w(req_heavy_w),
        .req_dryer_m(req_dryer_m), .req_delicate_w(req_delicate_w)
    );
```

```
always #5 clk_sys_mhz = ~clk_sys_mhz;


initial begin

    clk_sys_mhz = 0;
    rstt_sys_n = 1;
    pls_qw = 0; pls_hw = 0; pls_dr = 0; pls_dw = 0;


    #50;
    rstt_sys_n = 0;


    #50;


    @(posedge clk_sys_mhz);
    pls_qw = 1;
    @(posedge clk_sys_mhz);
    pls_qw = 0;


    #100;
```

```
        @(posedge clk_sys_mhz);
        pls_hw = 1;
        @(posedge clk_sys_mhz);
        pls_hw = 0;

        #100;



        @(posedge clk_sys_mhz);
        pls_dr = 1; pls_dw = 1;
        @(posedge clk_sys_mhz);
        pls_dr = 0; pls_dw = 0;

        #100;



        #200;
        $finish;
    end

endmodule
```

Testbench 2

```
`timescale 1ns / 1ps

module tb_FSM_LndryMachine();

    logic clk_sys_mhz;
    logic rstt_sys_n;
    logic pls_qw, pls_hw, pls_dr, pls_dw;
    logic req_quick_w, req_heavy_w, req_dryer_m, req_delicate_w;

    FSM_LndryMachine uut (
        .clk_sys_mhz(clk_sys_mhz),
        .rstt_sys_n(rstt_sys_n),
        .pls_qw(pls_qw), .pls_hw(pls_hw), .pls_dr(pls_dr), .pls_dw(pls_dw),
        .req_quick_w(req_quick_w), .req_heavy_w(req_heavy_w),
        .req_dryer_m(req_dryer_m), .req_delicate_w(req_delicate_w)
    );

    always #5 clk_sys_mhz = ~clk_sys_mhz;

    initial begin
        clk_sys_mhz = 0;
        rstt_sys_n = 1;
        pls_qw = 0; pls_hw = 0; pls_dr = 0; pls_dw = 0;

        #20 rstt_sys_n = 0;
```

```
$display("FSM TEST STARTED");

@(posedge clk_sys_mhz);
pls_qw = 1;
@(posedge clk_sys_mhz);
pls_qw = 0;

#1;
if (req_quick_w === 1'b1)
   $display("Quick Wash request handled correctly.");
else
   $display("Quick Wash request failed!");

@(posedge clk_sys_mhz);
#1;
if (req_quick_w === 1'b0)
   $display("Returned to IDLE state.");
else
   $display("FSM failed to return to IDLE!");

#50;

@(posedge clk_sys_mhz);
pls_hw = 1; pls_dr = 1;
@(posedge clk_sys_mhz);
pls_hw = 0; pls_dr = 0;
```

```
        #1;
        if (req_heavy_w === 1'b1 && req_dryer_m === 1'b0)
            $display("Priority rule passed.");
        else
            $display("Priority rule violated!");

        #50;

        @(posedge clk_sys_mhz);
        pls_qw = 1; pls_hw = 1; pls_dr = 1; pls_dw = 1;
        @(posedge clk_sys_mhz);
        pls_qw = 0; pls_hw = 0; pls_dr = 0; pls_dw = 0;

        #1;
        if (req_quick_w === 1'b1 && req_delicate_w === 1'b0)
            $display("Full load priority passed.");
        else
            $display("Full load priority failed!");

        #50;
        $display("ALL TESTS COMPLETED.");
        $finish;
    end

endmodule
```